

Reading a TypeScript Declaration

Every variation of `const o: { a?: number } = {}` — the three slots and what can go in each

October 2026

The shape

Every declaration has three slots. Only the first and last are required; the middle one (the type) is TypeScript's addition and runs nothing.

```
const o : { a?: number } = {} ;
 1     2     3             4
```

1. **keyword**: `const` (cannot rebind), `let` (can rebind), `var` (old, avoid).
2. **name**: the label.
3. **: TYPE**: what the compiler will allow in this label. Erased at runtime. Optional; omitted, TS infers it from the value.
4. **= VALUE**: the runtime value. Optional with `let` (then the value is `undefined`).

Python parallel: `o: dict = {}` has the same three slots; Python ignores the hint, TS enforces it.

Slot 1: the keyword

Form	Meaning	Python
<code>const x = 1</code>	label fixed; contents of an object/array still mutable	no equivalent (convention: never rebind)
<code>let x = 1</code>	label can be reassigned later	ordinary <code>x = 1</code>
<code>let x;</code>	declared, value <code>undefined</code> , type <code>any</code> unless annotated	<code>x = None</code>
<code>let x: number;</code>	declared with a type, no value yet; TS errors if read before assignment	
<code>var x = 1</code>	function-scoped, hoisted; smell in modern code	

Slot 3: the type, every form you will meet

Written	Reads as	Example value
<code>: number</code>	a number	<code>1, 2.5, NaN</code>
<code>: string</code>	a string	<code>"a"</code>
<code>: boolean</code>	<code>true</code> or <code>false</code>	<code>true</code>
<code>: null / : undefined</code>	exactly that value	
<code>: any</code>	anything, checking off	× in review
<code>: unknown</code>	anything, must narrow before use	✓ for external data
<code>: void</code>	returns nothing useful (functions)	
<code>: never</code>	never returns (throws, loops forever)	
<code>: number[]</code>	array of numbers	<code>[1, 2]</code>
<code>: Array<number></code>	same as <code>number[]</code>	<code>[1, 2]</code>
<code>: string[][]</code>	array of arrays of strings	<code>[["a"], ["b", "c"]]</code>
<code>: [string, number]</code>	tuple: exactly a string then a number	<code>["a", 1]</code>
<code>: { a: number }</code>	object with REQUIRED key <code>a</code>	<code>{ a: 1 }</code>
<code>: { a?: number }</code>	object with OPTIONAL key <code>a</code> (<code>number undefined</code>)	<code>{}</code> or <code>{ a: 1 }</code>
<code>: { a: number; b?: string }</code>	two keys; <code>;</code> or <code>,</code> separates them	<code>{ a: 1 }</code>
<code>: { readonly a: number }</code>	key cannot be reassigned (compile-time only)	
<code>: { [key: string]: number }</code>	any string key, number values (index signature)	<code>{ x: 1, y: 2 }</code>
<code>: Record<string, number></code>	same as the index signature above	<code>{ x: 1 }</code>
<code>: Map<string, number></code>	a Map from strings to numbers	<code>new Map()</code>
<code>: Set<string></code>	a Set of strings	<code>new Set()</code>
<code>: string number</code>	union: either	<code>"a"</code> or <code>1</code>
<code>: string null</code>	a string or null	<code>null</code>
<code>: "a" "b"</code>	literal union: only those two strings	<code>"a"</code>
<code>: A & B</code>	intersection: has everything in A and B	

Written	Reads as	Example value
<code>: User</code>	a named interface/type declared elsewhere	
<code>: User[]</code>	array of that named type	
<code>: Partial<User></code>	<code>User</code> with every key optional	<code>{}</code>
<code>: Required<User></code>	<code>User</code> with every key required	
<code>: Pick<User, "id"></code>	only the <code>id</code> key of <code>User</code>	
<code>: Omit<User, "id"></code>	<code>User</code> without <code>id</code>	
<code>: Promise<User></code>	a promise that resolves to a <code>User</code>	<code>fetchUser()</code>
<code>: () => void</code>	a function taking nothing, returning nothing	<code>() => {}</code>
<code>: (x: number) => string</code>	a function from number to string	<code>(x) => String(x)</code>
<code>: (...args: unknown[]) => void</code>	a function taking any arguments	
<code>: typeof DEFAULTS</code>	the type of an existing value	
<code>: keyof User</code>	union of <code>User</code> 's key names	<code>"id" "name"</code>

Slot 4: the value

Written	Reads as
<code>= {}</code>	empty object
<code>= { a: 1 }</code>	object with key <code>a</code>
<code>= { a }</code>	shorthand for <code>{ a: a }</code> (key and variable share a name)
<code>= { ...o, b: 2 }</code>	copy of <code>o</code> with <code>b</code> added or overridden
<code>= []</code>	empty array
<code>= [1, 2]</code>	array literal
<code>= [...a, ...b]</code>	concatenation
<code>= new Map() / = new Set()</code>	empty Map / Set
<code>= new Map([["k", 1]])</code>	Map seeded from pairs
<code>= (x) => x * 2</code>	arrow function
<code>= async () => { ... }</code>	async arrow, returns a Promise
<code>= await f()</code>	the unwrapped result of a Promise (inside <code>async</code>)
<code>= f()</code>	whatever <code>f</code> returns; if <code>f</code> is async this is a Promise $\times$
<code>= cond ? a : b</code>	ternary
<code>= x ?? 10</code>	<code>x</code> , or 10 if <code>x</code> is null/undefined
<code>= x 10</code>	<code>x</code> , or 10 if <code>x</code> is falsy
<code>= x as User</code>	cast; no runtime check
<code>= x!</code>	assert non-null; no runtime check
<code>= value as const</code>	freeze literals into literal types
<code>= value satisfies User</code>	check against <code>User</code> without widening

Mixed: destructuring in slot 2

Written	Reads as	Python
<code>const { a, b } = o</code>	pull <code>o.a</code> and <code>o.b</code> into <code>a</code> and <code>b</code>	<code>a, b = o["a"], o["b"]</code>
<code>const { a: x } = o</code>	pull <code>o.a</code> into a variable named <code>x</code>	
<code>const { a = 5 } = o</code>	pull <code>o.a</code> , default 5 if undefined	<code>o.get("a", 5)</code>
<code>const { a, ...rest } = o</code>	<code>a</code> , and <code>rest</code> = the other keys	
<code>const [x, y] = arr</code>	first two elements	<code>x, y = arr[:2]</code>
<code>const [x, , z] = arr</code>	skip the second	
<code>const [head, ...tail] = arr</code>	first and the rest	<code>head, *tail = arr</code>
<code>const { a }: { a: number } = o</code>	destructure with a type annotation	
<code>for (const [k, v] of Object.entries(o))</code>	destructure each pair in a loop	<code>for k, v in o.items()</code>

Same slots in a function signature

```
function f(id: string, n: number = 0, opts?: Options): Promise<User> { ... }
```

~~~~~      ~~~~~      ~~~~~      ~~~~~

required    default value    optional      return type

| Written                                              | Reads as                                                                          |
|------------------------------------------------------|-----------------------------------------------------------------------------------|
| <code>id: string</code>                              | required parameter of type string                                                 |
| <code>n: number = 0</code>                           | parameter with default; type inferred if omitted ( <code>n = 0</code> )           |
| <code>opts?: Options</code>                          | optional parameter; inside, <code>opts</code> is <code>Options   undefined</code> |
| <code>...rest: number[]</code>                       | collect remaining arguments                                                       |
| <code>): Promise&lt;User&gt;</code>                  | return type; <code>async</code> functions always return a Promise                 |
| <code>): void</code>                                 | returns nothing                                                                   |
| <code>): x is Circle</code>                          | type guard: returns boolean, narrows <code>x</code>                               |
| <code>&lt;T&gt;(x: T): T</code>                      | generic: whatever type goes in comes out                                          |
| <code>const f = (x: number): string =&gt; ...</code> | arrow with param and return types                                                 |

## Same slots in a class

```
class A {
  private x = 1;           // field with inferred type number
  readonly id: string;     // field declared, assigned in constructor
  static count = 0;        // on the class, not instances
  constructor(public name: string) {} // parameter property: declares AND assigns this.name
  get total(): number { return 0 } // getter, read as a.total
}
```

## Decoding checklist

1. Find the `=`; everything left is the declaration, everything right is the runtime value.
2. On the left, find the `:`; everything after it up to `=` is the type and runs nothing.
3. In the type, `?` after a key means optional, `|` means or, `[]` after a type means array of, `<>` means generic parameter.
4. In the value, `{}` is an object, `[]` is an array, `=>` is a function, `new` builds an instance, `await` unwraps a Promise.
5. Ask: can the value at runtime actually be what the type says? `as`, `!`, `any`, and `JSON.parse` are the places the answer can be no.